

Session 3

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 14-Sep-2026

Announcements

Previously on CRB1

- Session 2: Refresher Module 1 (PDF) — Python data model, containers, control flow
- Notebooks 1–2 (NumPy arrays, indexing, broadcasting, vectorization)

Questions?

Today's Agenda

- Refresher Module 1 (PDF): the tabular model (Pandas) and the PyTorch bridge
- Notebook 3 — Pandas for Bioengineers
 - Series/DataFrame, selecting, groupby, merge, the NumPy bridge
- Notebook 4 — Matplotlib for Bioengineers
 - figure/axes model, common plot types (Colab writes the code)
- Kahoot

The Scientific Python Stack

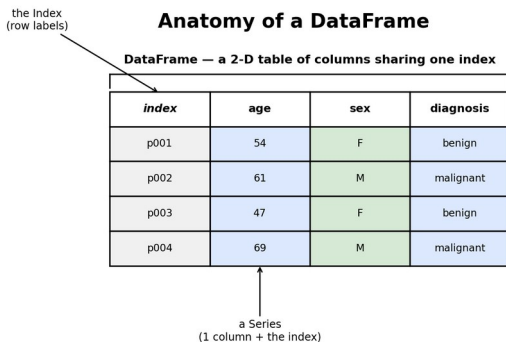
Where today's session fits in the ML workflow

- NumPy — the numerical array; the unit of scientific computing in Python (Session 2)
 - Pandas — labelled tables for loading and cleaning data (today)
 - Matplotlib — plotting and visualisation (today)
 - Scikit-Learn — classical ML models, one uniform interface: fit / predict / transform
 - PyTorch — tensors, autograd and neural networks (deep-learning half of the course)
-
- The ML workflow: Get data → Explore → Prepare → Train → Evaluate → iterate

Pandas

Labelled, tabular data structures built on top of NumPy

- Series (1-D) and DataFrame (2-D): the tabular model
- The Index is not just a row number — it aligns data automatically
- Each column is backed by a NumPy array under the hood
- `head()` / `info()` / `describe()` — the first three calls on any new table
- Loading a CSV/table into a DataFrame is usually step 1 of any analysis
- Split-apply-combine (`groupby`) replaces per-group loops
- Benefits: labelled access, automatic alignment, vectorized column ops

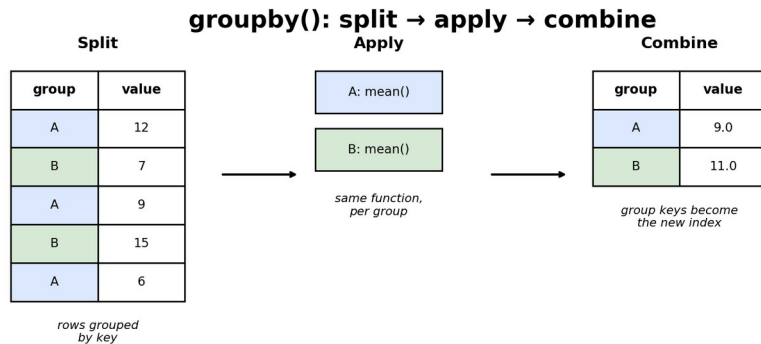


Every column is a Series; the shared Index aligns rows across all of them.

Pandas: Selecting, Cleaning & Combining

Getting the rows/columns you want, and joining tables together

- `[]` for columns, `.loc` for labels, `.iloc` for position — same boolean masks as NumPy
- Missing data: NaN propagates — `isna()` / `fillna()` / `dropna()` decide what to do
- `merge()` / `concat()` join tables the way SQL joins or stacks tables
- Bridge back to NumPy: `.to_numpy()` when you need a raw array (e.g. for PyTorch)



`df.groupby("group")["value"].mean()` — one line replaces the loop.

Let's practice — Notebook 3

https://github.com/SergiSama/UIC-CRB1-2026-2027/blob/main/m1_pandas_student.ipynb

[Open Notebook 3 in Google Colab](#) →

The PyTorch Bridge

Refresher Module 1 (PDF), §5 — where Pandas/NumPy data meets PyTorch

- Tensors extend arrays with three things: device placement (CPU/GPU), autograd, and a batch-first shape convention
 - same data model as NumPy — indexing, broadcasting, reductions all carry over
- `torch.from_numpy(arr)` shares memory with the array — no copy, edits propagate both ways
- Two practical gotchas: cast to float32 (NumPy defaults to float64), and add the batch axis models expect
- Shapes follow batch-first: (batch, features) for tables, (batch, channels, height, width) for images
- Going back: `tensor.numpy()` once autograd is detached (`.detach()`)
- This is the last hand-off before Module 1 data reaches a neural network

A Worked Thread

Refresher Module 1 (PDF), §6 — load, explore, prepare, bridge

- Load — read a CSV into a DataFrame; head() / info() first, always
 - a quick df[col].hist() shows the shape of what you just loaded
- Explore — groupby(...).describe() compares patients across a category
- Prepare — standardize with $(X - \mu) / \sigma$, NumPy broadcasting under the hood
- Bridge — torch.from_numpy(X_scaled) hands the array to a model
- Matplotlib, lightly: Colab already suggests the common plot types (line, scatter, histogram, bar) and writes the plt/df.plot() code for you
- The pipeline in one line: DataFrame → NumPy array → tensor

Let's practice — Notebook 4

Colab already suggests this notebook's plots and writes the code for you

https://github.com/SergiSama/UIC-CRB1-2026-2027/blob/main/m1_matplotlib_student.ipynb

[Open Notebook 4 in Google Colab](#) →

Before Next Session

- Finish both notebooks' exercises (solution cells included — check your own work)
- Re-read Refresher Module 1 (PDF) sections 4-6 if you haven't already
- Session 4 preview: Linear Algebra (Module 2 PDF)

Q & A



End of Session 3

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 14-Sep-2026